

Análise de Código Java

Longest Common Subsequence (LCS)

Código Analisado

```
public class LCS {
    public static int lcs(String a, String b) {
        int m = a.length(), n = b.length();
        int[][] dp = new int[m + 1][n + 1];
        for (int i = 1; i <= m; i++)
            for (int j = 1; j <= n; j++)
                dp[i][j] = a.charAt(i-1) == b.charAt(j-1)
                    ? dp[i-1][j-1] + 1
                    : Math.max(dp[i-1][j], dp[i][j-1]);
        return dp[m][n];
    }

    public static void main(String[] args) {
        System.out.println(lcs("ABCBADAB", "BDCABA")); // 4
        System.out.println(lcs("AGGTAB", "GXTXAYB")); // 4
        System.out.println(lcs("", "ABC")); // 0
    }
}
```

Explicação

Objetivo Geral

A classe **LCS** calcula o comprimento da **Subsequência Comum Mais Longa** (*Longest Common Subsequence*) entre duas strings. Trata-se de um problema clássico de otimização combinatória, amplamente utilizado em comparação de textos, controle de versões (como o *diff* do Git) e bioinformática para alinhamento de sequências genéticas.

Algoritmo e Lógica Central

O método **lcs()** emprega **programação dinâmica** por tabulação (*bottom-up*). Uma matriz bidimensional **dp[m+1][n+1]** é preenchida iterativamente: se os caracteres nas posições *i* e *j* forem iguais, o valor é $dp[i-1][j-1] + 1$; caso contrário, é o máximo entre $dp[i-1][j]$ e $dp[i][j-1]$. A resposta final encontra-se em $dp[m][n]$.

Conceitos e Recursos Java

O código utiliza **métodos estáticos**, **arrays bidimensionais primitivos** (`int[][]`), acesso a caracteres por **charAt()**, o operador ternário para lógica condicional compacta, e o método utilitário **Math.max()**. A ausência de estruturas de dados complexas mantém o código enxuto e eficiente.

Complexidade

A complexidade de **tempo** é $O(m \times n)$, onde m e n são os comprimentos das strings de entrada, pois cada célula da matriz é preenchida exatamente uma vez. A complexidade de **espaço** também é $O(m \times n)$ devido à alocação integral da tabela de memoização.

Decisões de Projeto e Limitações

A abordagem iterativa evita o risco de *stack overflow* presente na versão recursiva com memoização. Uma limitação relevante é o alto consumo de memória para strings muito longas; isso pode ser mitigado com a técnica de **linha rolante** (*rolling array*), que reduz o espaço para $O(n)$. Além disso, o método retorna apenas o **comprimento** da LCS, não a subsequência em si.